SUBJECT

Agent Based Modeling

CP1 · TASK 1 & TASK 2

DayMate: AI Daily Diary & Reminder Assistant

SUBMITTED BY	Ahmed Ali
ROLL NUMBER	L1F23BSAI0092
SUBMITTED TO	Sir Zeeshan Hussain

Table of Contents

1. Purpose **4**

2. Scope **4**

3. Architecture **4**

4. How a chat message works **5**

4.1 MCP tools 6

5. How reminders are delivered **6**

6. Data model **7**

7. Design rules **7**

8. Verification **7**

9. Running it **8**

10. n8n **9**

11. Results **10**

Notification or Remainders **11**

Mobile Notification **12**

DayMate: AI Daily Diary & Reminder Assistant

A Telegram assistant that logs your day, sets reminders and writes summaries. An LLM agent works through tools on a Python MCP server, and n8n runs the scheduled automation.

Item	Detail
Language	Python 3.12
Protocol	MCP (FastMCP, streamable HTTP)
LLM	Groq · gpt-oss-120b
Database	SQLite
Automation	n8n 2.45 in Docker
Messaging	Telegram Bot API
Date	November 2026

1. Purpose

People forget to record what they did and lose track of small deadlines. DayMate keeps both in one chat: you type what you did or what you need to remember, and the assistant stores it, reminds you on time and summarises your day or week from what was actually recorded.

For the assignment it demonstrates an **MCP server whose tools are called by an AI agent**, a **real external API** (Open-Meteo weather) and a working **n8n agent workflow**.

2. Scope

In scope	Out of scope
• Diary: add, view by date or range, update entries • Reminders: create, list, cancel, delivered on time • Daily and weekly summaries built from stored entries • Weather: current, 15-day forecast, historical • Per-user timezone and per-user data isolation	• Web or mobile dashboard • Calendar sync, voice or image input • Recurring reminders • Long-term semantic memory or RAG • Cloud hosting (runs locally)

3. Architecture

One Python server is the single source of truth. The agent reaches it over MCP, and n8n reaches it over a small REST API on the same port. Neither the model nor n8n touches the database directly.

Layer	Responsibility
`app/messaging`	Telegram adapter. Turns updates into plain messages and sends replies. No business logic.
`app/agent`	System prompt, Groq client and the loop that lets the model call MCP tools until it can answer.
`app/mcp_server`	FastMCP tools plus the REST endpoints for n8n. A bearer token protects both.
`app/services`	Diary, reminders, summaries, weather and delivery rules. All validation lives here.
`app/database`	SQLite schema and SQL. Every user-owned query filters on `user_id`.
`n8n/`	Three exported workflows: reminder delivery, daily summary, weekly report.

Table 1. Code layers. Figure 1 below shows how they connect.

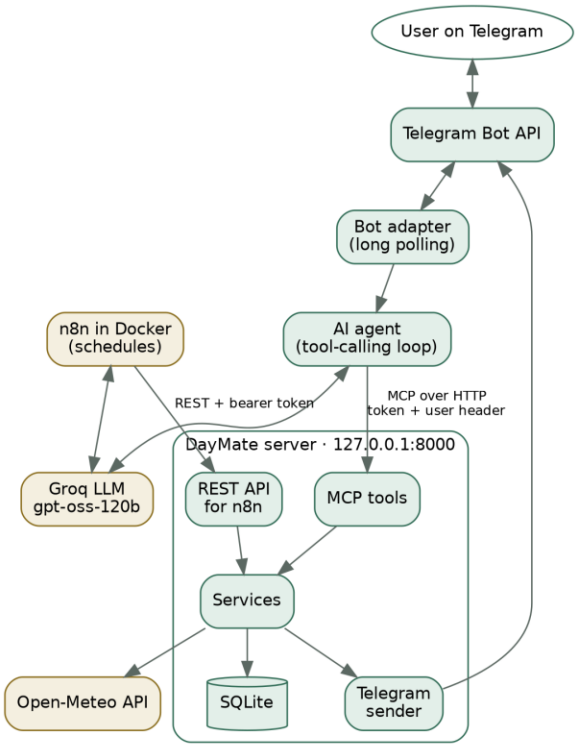


Figure 1. Components and trust boundaries. n8n never calls Telegram itself; the server sends every message.

4. How a chat message works

Example: the user sends “Remind me tomorrow at 10 AM to submit my assignment”.

#	From → To	Action
1	User → Bot	Sends the message text.
2	Bot → Agent	Passes the text and the internal user id.
3	Agent → Groq LLM	Sends the prompt (local time, timezone) and the tool list.
4	LLM → Agent	Requests `create_reminder(message, scheduled_at)`.
5	Agent → MCP server	MCP call with headers: token and `X-DayMate-User-Id`.
6	MCP server → SQLite	Validates, converts to UTC and inserts the reminder.
7	MCP server → Agent	Returns success: Tue 10:00 (Asia/Karachi).
8	Agent → LLM	Sends the tool result; the LLM writes the final reply.
9	Agent → User	“Your reminder is set for Tue 10:00 (Asia/Karachi)”.

Table 2. The model chooses the tool and its arguments. The user's identity comes from the bot, never from the model.

4.1 MCP tools

Tool	What it does
`add_diary_entry`	Saves an activity for a local date
`get_diary_entries`	Lists entries for a date or range, with optional keyword search
`update_diary_entry`	Edits an entry by id
`create_reminder`	Schedules a reminder in the user's timezone and rejects past times
`list_reminders`	Shows pending reminders, or reminders with any status
`delete_reminder`	Cancels a reminder (kept with status `cancelled`)
`get_daily_summary`	Returns one day's entries and the counts computed from them
`get_weekly_summary`	Returns the last 7 days of entries with computed counts
`get_weather`	Gets real Open-Meteo data: current, forecast or historical
`set_timezone`	Saves the user's IANA timezone

5. How reminders are delivered

n8n is the only scheduler. Every minute it leases the reminders that are due, then asks the server to deliver each one. The server sends the Telegram message and records the result, so a reminder is marked `sent` only after Telegram confirms it.

#	From → To	Action
1	n8n → Server	`POST /api/reminders/claim-due` (runs every minute).
2	Server → SQLite	Leases due pending reminders for 5 minutes.
3	Server → n8n	Returns the leased reminders.
4	n8n → Server	For each reminder: `POST /api/reminders/{id}/deliver`.
5	Server → Telegram	`sendMessage`.
6a	If Telegram confirms	Server sets `status = sent` and returns 200 Delivered.
6b	If the send fails	Server releases the lease, records the error, adds 1 to the attempt count and returns 502 (will retry; failed after 5 attempts).

Table 3. Reminder delivery sequence.

- **No duplicates:** a leased reminder is invisible to overlapping runs. A repeated delivery call finds it already sent and sends nothing.
- **Retries:** failures stay `pending` and are retried up to 5 times, then become `failed`.
- **Why the server sends:** Telegram is unreliable from inside Docker on this network, and n8n then needs no Telegram credential.

The daily (23:00) and weekly (Sunday 20:00) workflows ask the server which users are due in their own timezone. An n8n AI Agent node with the Groq model writes the summary from the returned entries, and the server sends it.

6. Data model

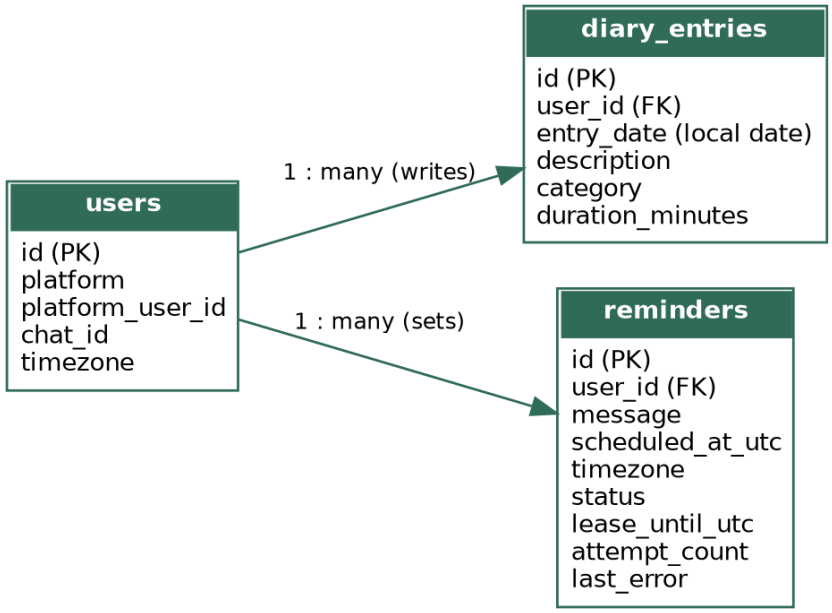


Figure 2. Entity relationships. Reminder status is one of pending, sent, cancelled or failed.

7. Design rules

- **Identity:** no tool accepts a `user_id`. Our code sets the user in a request header, so the model cannot read or change another user's data.
- **Truthfulness:** the agent confirms an action only after the tool reports success. Summary counts and totals are computed in code, and the LLM writes only the prose.
- **Time:** reminders are stored in UTC with the user's timezone and shown in local time. Vague times such as “tomorrow morning” make the agent ask for an exact time.
- **Secrets:** keys live in `.env` and in n8n's encrypted credential store. The API listens on localhost behind a bearer token.

8. Verification

Check	Result	Status
Automated tests	52 tests: services, timezones and DST, user isolation, leases and retries, MCP over HTTP, agent loop, Groq adapter, Telegram sender	**Pass**
Live Groq agent	9 scenarios through the real model and real MCP tools: diary add, view and update; reminder create, list and cancel; vague-time clarification; daily and weekly summaries	**Pass**

Check	Result	Status
Open-Meteo	Current, forecast and historical endpoints return real data	**Pass**
Reminder delivery	n8n delivered an overdue reminder and later ones to Telegram; each was marked sent once and a re-run sent nothing	**Pass**
Summary workflows	Ran end to end in n8n; the AI Agent produced a grounded report	**Pass**
Summary schedules	Daily and weekly workflows are built but not yet switched on	**Inactive**

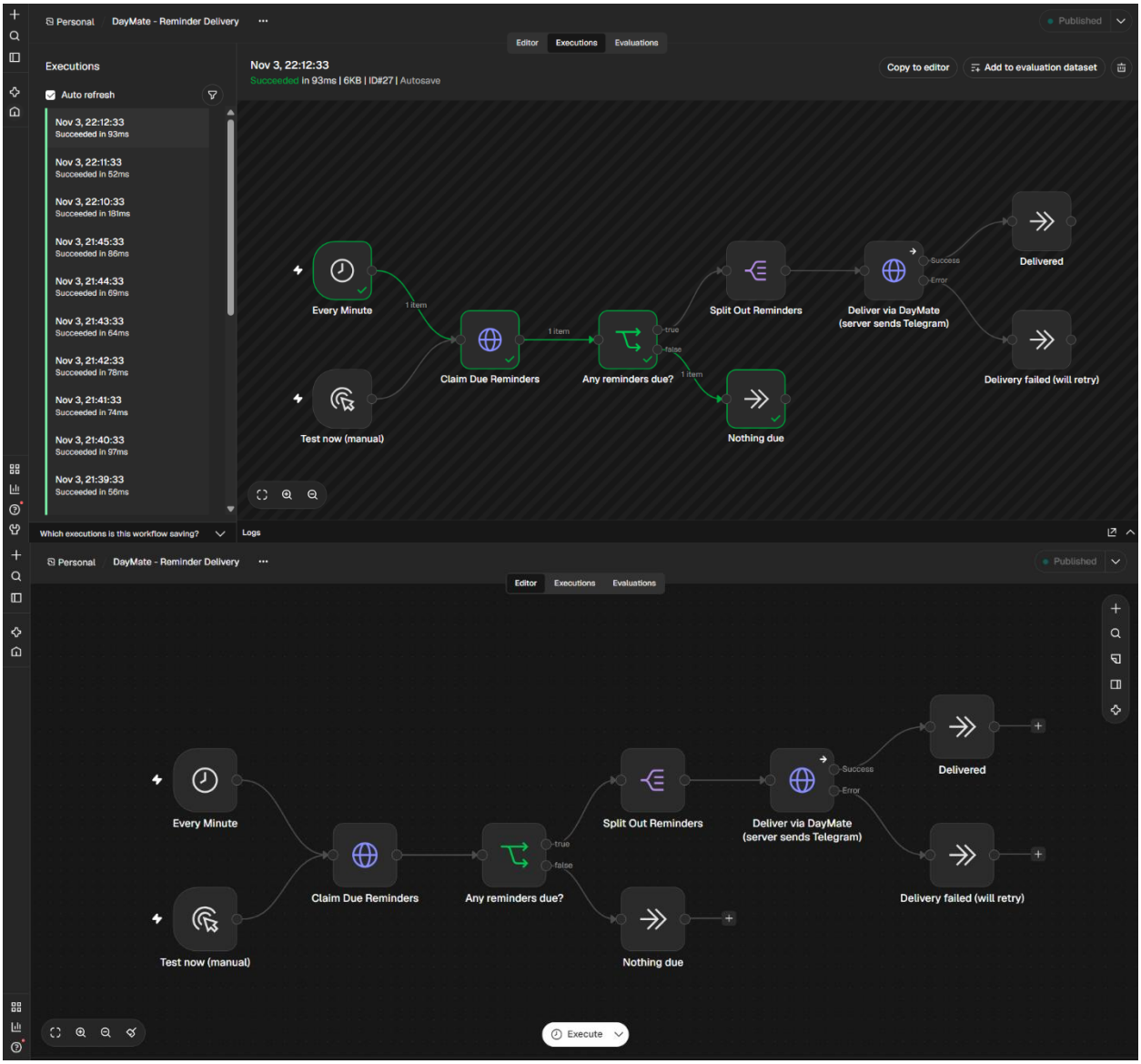
9. Running it

Prerequisites: Docker Desktop running and `.env` filled in (`GROQ_API_KEY`, `TELEGRAM_BOT_TOKEN`, `MCP_AUTH_TOKEN`).

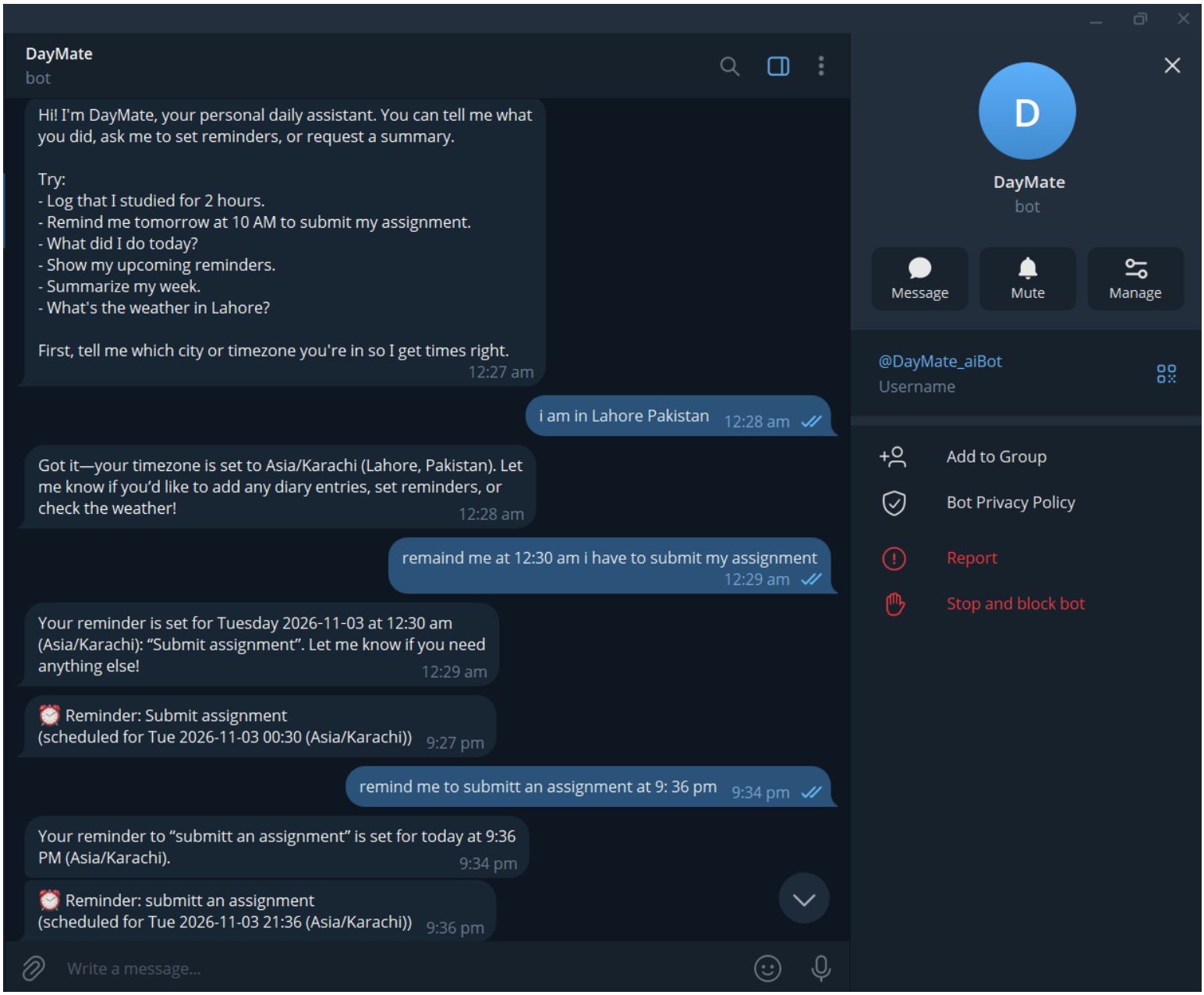
```
D:\DayMate\start_daymate.bat     # starts n8n, the server and the Telegram bot
D:\DayMate\stop_daymate.bat      # stops everything; data is kept
```

Then message `@DayMate_aiBot` on Telegram. n8n runs at http://localhost:5678.

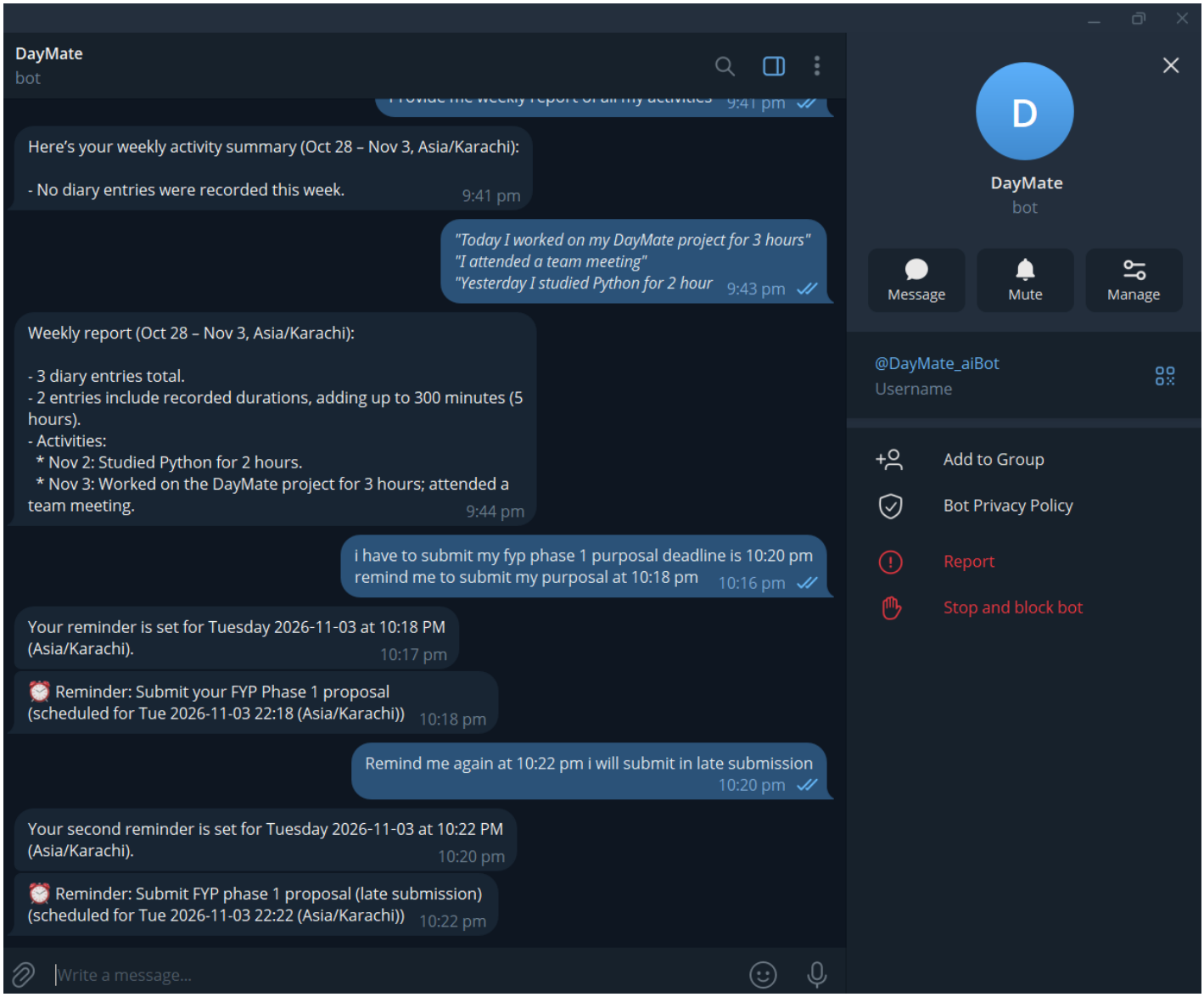
10. n8n

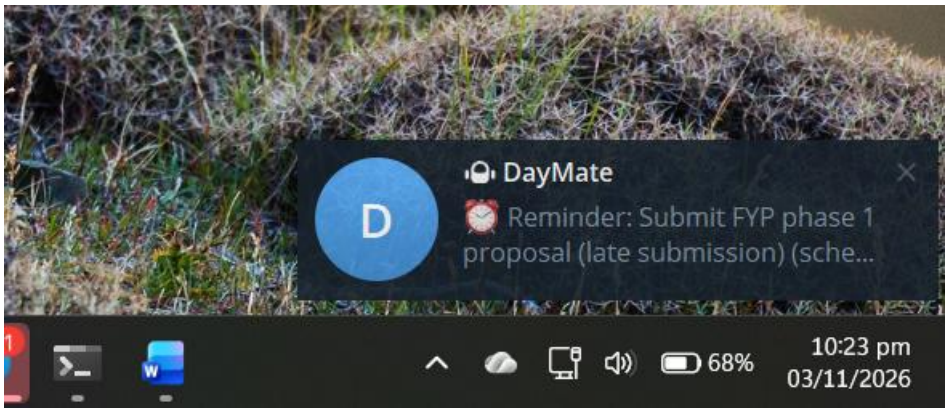

Personal
DayMate - Reminder Delivery
Published
Editor
Executions
Evaluations
Executions
Auto refresh
Nov 3, 22:12:33
Succeeded in 93ms
Nov 3, 22:11:33
Succeeded in 52ms
Nov 3, 22:10:33
Succeeded in 181ms
Nov 3, 21:45:33
Succeeded in 86ms
Nov 3, 21:44:33
Succeeded in 69ms
Nov 3, 21:43:33
Succeeded in 64ms
Nov 3, 21:42:33
Succeeded in 78ms
Nov 3, 21:41:33
Succeeded in 74ms
Nov 3, 21:40:33
Succeeded in 97ms
Nov 3, 21:39:33
Succeeded in 56ms
Nov 3, 22:12:33
Succeeded in 93ms | 6KB | ID#27 | Autosave
Copy to editor
Add to evaluation dataset
Every Minute
Test now (manual)
Claim Due Reminders
Any reminders due?
Split Out Reminders
Deliver via DayMate (server sends Telegram)
Delivered
Delivery failed (will retry)
Nothing due
1 item
true
false
Success
Error
Which executions is this workflow saving?
Logs
Execute

11. Results


DayMate
bot
Hi! I'm DayMate, your personal daily assistant. You can tell me what you did, ask me to set reminders, or request a summary.
Try:
- Log that I studied for 2 hours.
- Remind me tomorrow at 10 AM to submit my assignment.
- What did I do today?
- Show my upcoming reminders.
- Summarize my week.
- What's the weather in Lahore?
First, tell me which city or timezone you're in so I get times right.
12:27 am
i am in Lahore Pakistan 12:28 am
Got it—your timezone is set to Asia/Karachi (Lahore, Pakistan). Let me know if you'd like to add any diary entries, set reminders, or check the weather!
12:28 am
remaind me at 12:30 am i have to submit my assignment
12:29 am
Your reminder is set for Tuesday 2026-11-03 at 12:30 am (Asia/Karachi): "Submit assignment". Let me know if you need anything else!
12:29 am
⏰ Reminder: Submit assignment
(scheduled for Tue 2026-11-03 00:30 (Asia/Karachi)) 9:27 pm
remind me to submitt an assignment at 9: 36 pm 9:34 pm
Your reminder to "submitt an assignment" is set for today at 9:36 PM (Asia/Karachi).
9:34 pm
⏰ Reminder: submitt an assignment
(scheduled for Tue 2026-11-03 21:36 (Asia/Karachi)) 9:36 pm
Write a message...
D
DayMate
bot
Message
Mute
Manage
@DayMate_aiBot
Username
Add to Group
Bot Privacy Policy
Report
Stop and block bot

Notification or Remainders





Mobile Notification

